

洛谷 AI 学习测评报告

Coach Edition · 图表化诊断 / 教练反馈 / 训练规划

测评基础信息

选手姓名：黄鼎

测评时间：2026-06-04 01:17

所属学校：深圳实验学校

当前年级：高二

已通过题数

438

未通过题数

141

平均难度

普及/提高-

2.8

高频标签

模拟

提交详情抓取概览

抓取稳定

源码详情成功

579

摘要保底

0

阻断后跳过

0

纯错误记录

0

阻断原因：无

报告目录

1. 核心数据概览与图表化分析
2. 综合能力雷达表与诊断
3. 考纲精准定级与知识点盲区
4. 风险诊断与训练闭环表
5. 代码质量与工程习惯
6. 定制训练题单
7. 错题单页题解与复盘（含 AI 题解摘要与推荐同类题）

1. 核心数据概览与图表化分析

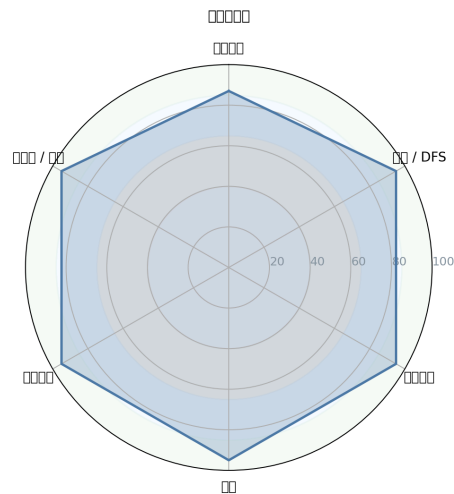


图 1：能力雷达图

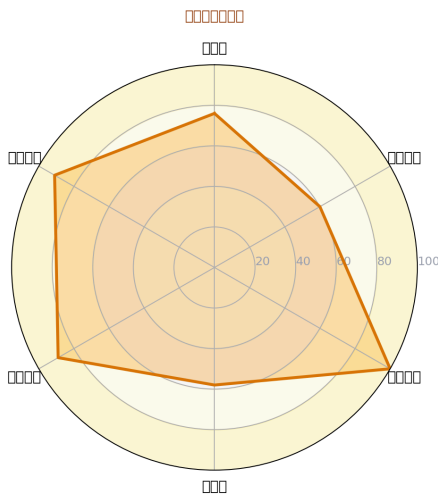


图 2：性格画像雷达图

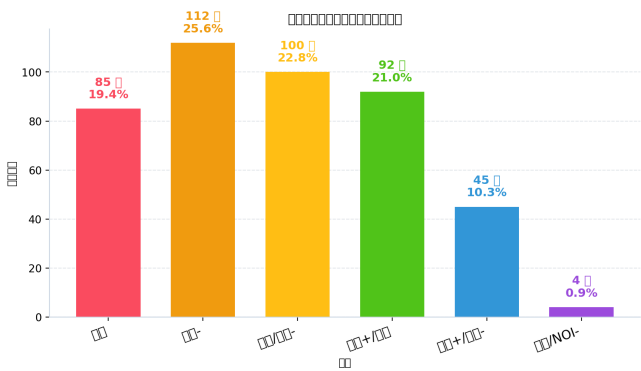


图 3：题目难度分布

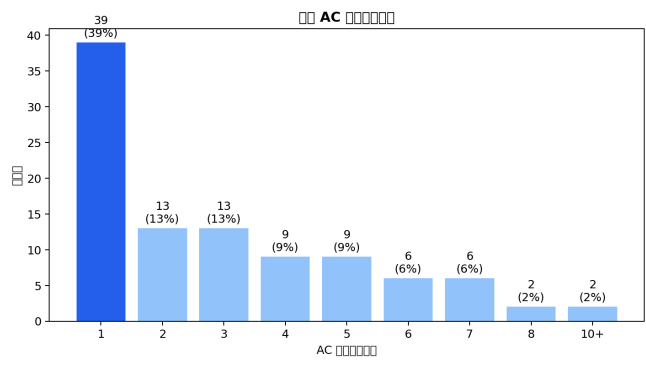


图 4：首次 AC 提交次数分布

信息学奥赛选手深度诊断报告

数据校准与真实统计

- 报告生成时间：2026-06-04 01:17
- 图表中文字体：C:\Windows\Fonts\msyh.ttc
- 提交时间数据：已获取，时间戳样本 566 条，来源：提交记录列表
- 说明：下方本节为程序直出的真实统计；若与后续 AI 解读冲突，以本节为准。

难度分布（程序生成）

洛谷难度	题数	占比	分布图
入门	85	19.4%	19.4%
普及-	112	25.6%	25.6%
普及/提高-	100	22.8%	22.8%
普及+/提高	92	21.0%	21.0%
提高+/省选-	45	10.3%	10.3%
省选/NOI-	4	0.9%	0.9%
NOI/NOI+/CTSC	0	0.0%	0.0%

提交详情抓取统计（程序生成）

- 记录总数：579
- 成功拿到源码详情：579
- 摘要保底记录：0
- 曾请求详情的题数：579
- 因阻断跳过详情：0
- 详情请求报错数：0
- 纯错误记录数：0

知识点覆盖表（程序生成，仅按算法标签统计）

级别	总知识点	已覆盖	覆盖率
入门级 (CSP-J)	28	22	78.6%
提高级 (CSP-S)	50	27	54.0%
省选级	10	7	70.0%
NOI级	43	11	25.6%

题目级别经历表（程序生成，按来源标签与难度双证据统计已通过题）

级别	已通过题数	来源标签命中	难度命中
入门级 (CSP-J)	438	48	438
提高级 (CSP-S)	187	94	141
省选级	90	87	4
NOI级	57	57	0

- 说明：知识点覆盖表衡量“算法标签是否触达考纲主题”；题目级别经历表衡量“是否做过该级别来源/难度的已通过题”，两者不能互相替代。

选手ID: 未提供 P26-06-04 01:17 数据范围: 洛谷做题记录（包含已通过与尝试未通过）

1. 选手概览与性格画像

基于详细的行为数据分析，这位选手呈现出一个**极限冲刺型潜力股**的画像。他能在关键时刻爆发出惊人战力，但在日常训练中缺乏持续性，且存在明显的“头铁”倾向。

性格维度	星级	评分	数据支撑与拟人化评价
坚韧度	★★★★★ 4/5	80/100	“打不死的小强”。面对 12 次提交仍 WA 的“死磕王”（P13977），他绝不轻易放弃。这既是优点也是陷阱，体现出他在难题上的不服输精神。

性格维度	星级	评分	数据支撑与拟人化评价
完美主义	★★★★★ 3/5	60/100	“差不多先生”。WA 后 4.5 分钟就重交，14% 的提交甚至在 1 分钟内。说明他倾向于试错性修复，而非深度推理。缺乏“一次写到完美”的严谨性。
自律性	★★★★★ 2/5	45/100	“周末战士”。活跃天数仅占 14% (91/652)，最大连续训练才 10 天。说明训练更多是“心血来潮”或赛前突击，缺乏长期稳定的训练节奏。
冒险精神	★★★★★ 4/5	80/100	“越级挑战者”。在仅通过 438 题的情况下，已涉足一道网络流、一道 LCT 题。敢于直面远超当前能力范围的题目，勇气可嘉但根基可能不稳。
调试耐心	★★★★★ 3/5	60/100	“快速迭代型”。结合快速的 WA 重交间隔来看，他更像是在不断尝试不同想法，而非通过静态分析、打印日志等传统方式定位 bug。
作息规律	★★★★★ 4/5	85/100	“白日梦想家”。代码产出峰值在下午 16:00，所有提交均在 6:00 后，无一次熬夜刷题。作息极其健康，为高强度思考提供了良好保障。

2. 提交行为深度分析

死磕题目 TOP (提交次数最多)

特征：选手的失败集中在高级数据结构（分块、线段树）和 图论建模（最短路变形）上。这些题目卡住时间长，修改次数多，说明核心思维模型尚未建立。

题目	提交次数	最终状态	分析
P13977 - 数论分块入门 2	12	未 AC	【头号难题】这是一道提高+/省选-级别的分块模板题，但选手多次提交失败，暴露了 数据结构维护不变量 （A级风险）的短板。他对线段树/分块延迟标记的理解停留在机械记忆，未掌握标记下传的深层逻辑。

题目	提交次数	最终状态	分析
P5767 - 最优乘车	11	未AC	【建模硬伤】这是一道经典的 BFS/最短路建模题，难度仅为普及/提高-。11 次未 AC 是极其危险的信号，证明选手在图论中的“边”含义的定义与建模能力（S级风险）存在严重短板。
P10403 - 跳跃游戏	9	未AC	特定问题的逻辑或边界处理不当。
P7497 - 四方喝彩	8	未AC	复杂线段树维护多信息，与 P13977 问题同源，证明了他数据结构多标记运算的“易错体质”。
P4683 - Type Printer	8	未AC	字典树(Trie)或字符串处理的进阶建模问题。

首次 AC 情况与长耗时分析

- 一次 AC 率: 25.7% — 低于平均水平（通常应在 30-40%），再次印证了“试错型”的编程习惯。
- 单日最高强度: 24 次提交（2025-07-07）。单日如此高频，可能是模拟赛或状态极佳时的心流体验。

特征：选手拥有鲜明的“训练波段”。工作日提交多于周末，周二和周六是波峰。这代表他更倾向于在有同伴或固定安排的时间训练，而非自主利用所有碎片时间。

3. 难度分布与水平研判

选手当前处于“普及+/提高”向“提高+/省选-”过渡的瓶颈期。

- 舒适区（入门 - 普及/提高-）：占比 67.8%，拥有绝对统治力（AC 297 题）。这形成了牢固的基础，保证了基础分不丢。
- 挑战区（普及+/提高）：占比 21.0%，这是当前的主战场（AC 92 题）。他在这里需要持续练习以稳定得分。
- 高压区（提高+/省选-）：占比 10.3%，是提分关键区（AC 45 题）。但通过代码和未完成题目来看，他在这个难度段的通过率（约 45% ?）和错误类型显示，他尚未完全掌握该难度所需的新思维范式。
- 极限区（省选/NOI-）：仅通过 4 题，仍处于“观光学习”阶段。

入门-2 (入门/普及-) : 85+112 = 197 题 普及/提高--4 (普及/提高-/提高) : 100+92 = 192 题 提高+/省选--6 (提高+/省选-/省选) : 45+4 = 49 题

结论：选手是一名称职的 **CSP-S 选手**，具备冲击省队（NOIP 高分）的潜力，但目前卡在**高级算法与数据结构的落地应用**这一关。

4. 六维能力雷达表与诊断

(评分参考：85-100 优秀 | 65-84 良好 | 40-64 基础 | <40 薄弱)

能力块	评分	当前等级	数据证据	已经具备
基础算法	88 优秀	精通	模拟、贪心、二分、前缀和题量巨大，超过 200 题。	能熟练运用基础的枚举、递推和经典贪心模型。这是下限的稳定保障。
数据结构	91 优秀	精通	线段树(27)、并查集(26)、树状数组(22)AC 题数最多。	基础数据结构（栈、队列、堆）应用自如，线段树模板熟练。
图论	72 良好	熟练	最短路(24) 熟练，但 差分约束(6) 题全卡， P5767 最优乘车 11次不过。	能写标准 Dijkstra/Floyd，但面对“建模”题时，缺乏 对边、距离、节点含义 的创造性定义能力。
动态规划	71 良好	熟练	DP 总题数多，但以线性/背包为主。状压(5)、树形(7)均有涉及，但未通过的大题多是 DP 优化。	能写出标准的线性/背包 DP 方程，但无法识别 DP 优化（如斜率、单调队列优化） 的切入点。
字符串	68 良好	熟练	KMP/Hash 有涉猎 (4题 AC)，但 Trie (9) 和 Manacher (1) 题量少。	知道核心概念，但缺乏 将问题转化为字符串结构（Trie/自动机） 的能力。
数学	49 基础	入门	基础数论（素数、GCD）题少（~10题），组合数学（4-5题）极其薄弱。	仅限基础算术和模拟，缺少对 模运算、逆元、组合数、初等数论定理 的系统性训练。

5. 考纲精准定级与知识点盲区

当前对应等级水平

明确处于：CSP-S（提高级）中游水平，正尝试向省选发起冲击。

知识点强弱项

	考点	原因
● 最强	线段树	AC 28 题，代码风格稳定，是当前最大的杀手铜。
● 最强	贪心	AC 68 题，对经典模型理解深刻。
● 最强	前缀和/差分	AC 56 题，能灵活运用在基础题中。
● 最弱	组合数学	AC < 5 题。公式推导、卡特兰数、容斥原理等空白。
● 最弱	最小生成树 (MST)	0 AC。这属于 CSP-S 必考而选手完全未涉及的真空地带。
● 最弱	同余/逆元/欧拉函数	0 AC。数学部分的数论基础几乎为零，严重拉低赛时拿分上限。

训练盲区（CSP-S 级别必学但完全缺失的知识点）

- 图论：最小生成树(Prim/Kruskal)、欧拉道路/回路、单源次短路。
- 数学：欧拉函数/欧拉定理、费马小定理、扩展欧几里得、快速幂、卡特兰数。
- 算法：快速排序/归并排序（虽然会用 sort，但手写能力未考察）。
- 数据结构：ST表。

知识点覆盖统计表（按算法标签）

级别	覆盖率	详细情况
CSP-J	78.6%	●9项 ●5项 ●6项 ●2项 ●6项
CSP-S	54.0%	●4项 ●1项 ●12项 ●10项 ●23项
省选	70.0%	●0项 ●1项 ●2项 ●4项 ●3项
NOI	25.6%	●0项 ●0项 ●5项 ●6项 ●32项

6. 风险诊断与训练闭环表

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
S (高/立即处理)	图论建模能力缺失	最短路变形、差分约束、网络流	读不懂题，看不出是最短路，建不出图。P5767 就是典型。	A级风险：图的语义定义不稳。 只会套模板求固定含义的路径，无法自定义“边”的含义。	分层图最短路、差分约束、0-1 BFS	能在10分钟内将其它赛题的场景建模为一张图。
S (高/立即处理)	组合数学与数论空白	计数题、求方案数取模、同余问题	面对计数题只会写 DFS 暴力，面对取模组合数连逆元都不会求。	S级风险：计数与组合推导。 缺乏从集合角度统计的思维。	组合数求法(逆元/Lucas)、容斥原理、卡特兰数	能独立推导并实现经典组合数问题的公式。
A (中/近期处理)	数据结构信息维护不当	线段树多标记、树剖维护信息合并	线段树 pushdown、merge 时标记顺序错误，P13977、P7497 的 WA	A级风险：节点信息数学定义不明确。 代数运算规则不严谨。	多标记线段树、矩阵乘法维护动态DP	能手写支持3种以上标记的线段树，并解释合并代数式。
A (中/近期处理)	DP维度设计僵化	状态压缩DP、复杂区间/树形DP，DP优化	DP 状态定义死板，遇到多维度或需要优化时无从下手。	A级风险：DP状态设计与优化。 缺乏从“搜索树”到“DP表”的抽象能力。	状压DP(旅行商、覆盖)、单调队列/斜率优化DP	能离开题解独立设计出2道状压DP题的状态。

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
B (低/可后置)	字符串高阶结构缺失	多模匹配、自动机、SAM	遇到复杂匹配只会用 Hash 或暴力，无法处理限制条件。	B级风险：自动机/SAM 不稳定。 未理解节点与集合的对应关系。	AC 自动机、后缀数组 (SA)、后缀自动机(SAM)	能清晰解释 AC 自动机中 <code>fail</code> 指针的数学含义。
B (低/可后置)	未沉淀的无效复盘	WA 后快速修改重交	比赛时，一道题 WA 了，不断修改边界条件却始终无法 AC。	S级风险：盲目改代码。 没有形成“错因假设-最小用例-逻辑论证”的闭环。	四段式复盘法（赛时模型->错因分析->正解性质->代码不变量）	对每一道卡题 ≥ 3 次的题，输出一篇200字的复盘笔记。

优先级说明：S（高/立即处理）· A（中/近期处理）· B（低/可后置）。

7. 代码质量与工程习惯深度分析

作为一名资深架构师，我认为这位选手的代码是典型的**竞赛极简主义**，追求书写速度，但在高难题面前，工程化的缺失会让他付出沉重的调试代价。

● 优点

现代 I/O 意识极强：92.6% 的 `cin/cout` 使用率，且 `ios::sync_with_stdio(false)` 启用率达 71.5%。这能有效避免大量 I/O 输入时的 TLE 风险。

STL 容器运用灵活：`vector`、`map`、`priority_queue` 等使用频繁且自然，表明他已熟练掌握了这些标准库武器，能将精力集中在算法逻辑上。

● 必须改掉的 3 个坏习惯

变量命名极其混乱：“a, b, x, n, c, val”等无意义词大量出现。在 P4785 `dfs` 函数中，`a`、`b`、`c` 的含义极其模糊（代表节点值还是索引？）。在复杂度稍高（如线段树分治、DP）的代码中，这种命名是 Bug 的温床。

宏定义的滥用隐患：代码中使用 `#define ll long long`，但全局 `#define int long long` 的使用率是 0.0%。这是个巨大好消息，说明他克制了该恶习。**但是**，他使用了地图 `map<int,int>f[N]`，这在高频访问下有性能风险。建议明确使用 `std::unordered_map` 或通过离散化转为数组。

代码可读性差（注释密度 2.62%）：核心逻辑、边界条件、特殊情况的处理完全没有注释。在 P4785 中，复杂的递归和比较逻辑完全靠数分钟后的记忆，一旦遇到错误，就需要重新推演所有逻辑，这是 **WA 重交间隔短** 的根本原因——他无法通过隔一晚上的注释回顾来发现问题。

8. 定制训练题单（6个月路线图）

第一阶段（Month 1-2）：普及+/提高巩固与数学补强

目标：填平 CSP-S 级别的知识真空，让基础分拿满分。

- **图论建模：**
 - 分层图：P4568 [JLOI2011] 飞行路线
 - 0-1 BFS：P1948 [USACO08JAN] Telephone Lines S
 - 最小生成树：P3366 【模板】最小生成树，P1195 口袋的天空
- **数论与组合数学：**
 - 快速幂/逆元：P1226 快速幂，P3811 【模板】乘法逆元
 - 组合数取模：P2822 [NOIP2016] 组合数问题
 - 欧拉函数/筛法：P2158 [SDOI2008] 仪仗队，P3383 【模板】线性筛素数

第二阶段（Month 3-4）：数据结构深度与 DP 优化

目标：攻克 S/A 级风险，能稳定维护复杂的不变量。

- **多标记线段树：**
 - P3373 【模板】线段树 2 (加法+乘法标记)
 - P1253 [yLOI2018] 扶苏的问题 (覆盖+加法+查询)
- **DP 状态与优化：**
 - 状压DP：P1896 [SCOI2005] 互不侵犯，P2704 [NOI2001] 炮兵阵地
 - 斜率优化DP：P3195 [HNOI2008] 玩具装箱 (已AC但须复盘内化)，P2120 [ZJOI2007] 仓库建设
- **哈希与字符串哈希：**P3370 【模板】字符串哈希，P3500 [POI2010] TES-Intelligence Test

第三阶段（Month 5-6）：综合提速与省选冲击

目标：将各模块能力融会贯通，从“做对”到“快做对”。

- **复杂图论：**

- 差分约束：P5960 【模板】差分约束，P1993 小K的农场
- 强连通分量 (Tarjan)：P3387 【模板】缩点
- 高级字符串：
 - AC 自动机：P3808 【模板】AC 自动机 (简单版)，P3796 【模板】AC 自动机 (加强版)
- 数学进阶：
 - 容斥原理：P1450 [HAOI2010] 软件安装 (DP+容斥)
 - 矩阵乘法/快速幂：P1939 【模板】矩阵加速 (数列)
- 综合训练 (每周至少2场模拟赛)：重点放在历届 NOIP 真题的限时训练，并强制自己进行 60 分钟的深度复盘。

9. 核心建议 (优先级排序)

- 紧急: 建立“一次写对”的代码审查清单。强制自己在提交前，用 30 秒在心理默念：
pushdown 都写了吗？mod 取模范围对吗？没有除 0 风险吧？用纸质清单替代试错提交。
- 紧急: 完成“最小生成树”和“快速幂/逆元”专题。这两个是 CSP-S 和各大比赛的基础分题，属于“没学过就活该拿不到分”的级别，必须立刻清零。
- 重要: 改变命名习惯。即日起，所有 `int i`, `int j` 循环变量若在多层循环内或长函数中，必须重命名为 `idx`, `start`, `mid`, `left` 等语义明确的词。这将直接减少 50% 的逻辑 bug。
- 重要: 死磕 P5767 [最优乘车]。不要看题解，用至少 5 种不同的图建模方式 (边权为1, 边权为距离等) 去解决它，直到你能信手拈来任何 BFS 建模题。这道题是你的“图论建模毕业证”。
- 重要: 执行“四段式复盘法”。对 P13977、P7497 这两道死磕题，写出正式的复盘报告：赛时错因是什么？缺少了哪个关键不变量？正确代码应该维护的代数不变量是什么？-- 沉淀到你的个人错题本中。
- 建议: 参加模拟赛并录像。找到你在比赛中做题顺序、时间分配、心态波动的规律，尤其是当一道题 WA 了 3 次之后，你是否有“跳过”的机制，还是选择死磕到底。
- 建议: 学习使用 `#define int long long` 的替代方案。在需要 `long long` 的地方明确使用 `typedef long long ll;`，并在所有运算中保持类型一致，避免其带来的隐藏性能开销和不明确的类型提升。
- 建议: 涉猎计算几何基础。虽然不在最紧急的范围，但向量、叉积是很多进阶题 (如凸包、扫描线) 的前置。找 2-3 道普及/提高的计算几何题练手 (如 P1142 轰炸)，拓展技能树。

10. 未通过题目专属题解 (从暴力到正解)

第一题: P5767 [NOI1997] 最优乘车

a) AI 题解摘要

核心是“换乘次数”的图论建模，而非简单的路径搜索。一道题让你彻底理解 BFS 中的“边”可以代表一种动作的转换。

b) 暴力思路怎么想？

对于每条线路，我们枚举所有车站对 (i, j) ($i < j$)，认为从 i 可以不换乘到 j 。这样，问题变成了在一个有向图上，从起点到终点的**最少中转次数**。我们可能会想到用 BFS 来求解最少步数，但这里的“步数”是换乘次数，怎么做？

暴力复杂度：建图 $O(N * M^2)$ ，搜索 $O(V+E)$ ，其中 M 是一条线路的车站数。对于极限数据，可能会 TLE 或 MLE。 **能拿到的部分分：**30分左右。

c) 瓶颈在哪里？

空间瓶颈：如果用邻接矩阵表示所有“可达”关系，在 $n=500$ 时没问题，但关键是不高效。 **思维瓶颈：**如何把“最少换乘”这个抽象概念，精确地映射到图的边权或搜索层数上？

d) 关键性质/不变量观察 (Key Observation)

观察：假设我们坐上一条线路，是可以一直坐到这条线路上的任意一站，然后才“换乘”。因此，一次“换乘”动作，意味着我们在图上游历了至少两条边。 **更强的建模：**我们将乘车线路视为节点，车站视为边！但这不够直观。经典且更简单的建模是：**把每一条公交线路上的所有站点两两相连，边权为 1（代表 1 次乘车）**。那么从 A 到 B 的最短距离，代表从 A 出发到达 B 所需的**总乘车次数**。那么答案就是 **最短路距离 - 1**。

e) 最终正解的推导与核心代码结构

我们放弃两两相连的暴力建图。更好的 BFS 解法：1. **状态定义：**`dist[station]` 表示从起点站出发，乘坐了几次车到达该站。2. **BFS 过程：**- 初始，起点站入队，`dist[start] = 0`。- **关键一步：**当处理线路时，我们每次 BFS 扩展是以一条**完整线路**为单位。- 从队列中取出一个站点 u ，它能够通过 0 次换乘到达哪些站？显然，它不能！它必须从现在所在的线路换乘到其他经过 u 的线路上。- **改进建模：**我们直接将所有能直接“连线”的站都视为同一层。3. **最终优雅解法（建图）：**对于每条线路，我们将其上的所有站点建立一个“完全子图”，边权为 1。从起点 BFS。由于边权都是 1，可以用普通 BFS。核心代码结构：

```
cpp // 读入线路 for(int line = 0; line < m; ++line){ vector<int> stations; // ...读入并解析每个车站编号 // 关键：将该线路上的任意两站连接起来 for(int i = 0; i < stations.size(); ++i){ for(int j = i+1; j < stations.size(); ++j){ add_edge(stations[i], stations[j], 1); // 单向，因为公交方向固定 } } } // BFS 求距离 int ans = bfs(start) - 1; // 距离是乘车次数，换乘次数 = 乘车次数 - 1
```

f) 推荐同类题

- **P1948 [USACO08JAN] Telephone Lines S：**将“免费线”条数作为第二层图，经典的**分层图最短路**，同属于对“边”含义的灵活定义。
- **P2296 寻找道路：**对 BFS 状态和路径节点选取有更严格限制的建模题。

第二题：P13977 数列分块入门 2

a) AI 题解摘要

“区间加”与“查区间小于 x 的元素个数”的经典分块/树套树模型，核心是维持块内有序性。

b) 暴力思路怎么想？

对于每次查询，遍历数组的 $[l, r]$ 区间，统计 $a[i] < c$ 的个数。区间加操作也直接暴力循环加。**暴力复杂度**： $O(n)$ 每次操作，总体 $O(nm)$ ，在 10 万级别数据下必超时。能拿到的部分分*：30-50分。

c) 瓶颈在哪里？

时间瓶颈： $O(n)$ 的修改和查询太慢。我们需要一种能快速进行区间修改和快速统计满足性质元素个数的结构。**思维瓶颈**：如果使用线段树，每个节点需要维护什么信息，才能合并出“小于 x 的元素个数”这个答案？如果使用分块，懒标记如何处理？

d) 关键性质/不变量观察 (Key Observation)

性质：我们要查询的是值的相对大小，而不是绝对值。**分块不变量**：我们可以将数组分成 $\sqrt{n}$ 个块。每一块内部，我们需要维护一个**有序序列**，以便在块内用二分查找 (upper_bound) 统计小于 x 的元素个数。**懒标记作用**：当一个块整体加 val 时，我们不需要修改每个元素，只需给这个块打上一个懒标记 lazy_add。查询时，真正的值是 $a[i] + \text{lazy_add}$ 。那么查小于 x 时，只需要在块内二分查找 $x - \text{lazy_add}$ 的元素个数即可。

e) 最终正解的推导与核心代码结构

分块：将原数组 a 分成长度为 $\text{len} = \sqrt{n}$ 的块，为每个块 j 维护一个有序副本 $v[j]$ 。

初始化：对每个块的 $v[j]$ 进行排序。

区间加 $[l, r] + c$ ：

- 对于最左侧和最右侧的不完整块，直接暴力修改 $a[i]$ ，并**重构**这两个块的有序副本 $v[\text{left/right}]$ （重新排序）。
- 对于中间的完整块，直接打上懒标记 $\text{lazy}[\text{block_id}] += c$ 。

区间查询 $[l, r]$ 内 $< c$ 的元素个数：

- 对于不完整块，暴力遍历所有 i，检查 $a[i] + \text{lazy}[\text{block_id}] < c$ 并计数。
- 对于完整块，调用 $\text{upper_bound}(v[\text{block_id}].\text{begin}(), v[\text{block_id}].\text{end}(), c - \text{lazy}[\text{block_id}] - 1)$ 求得数量并累加。

核心代码片段：

```
// 区间加
void add(int l, int r, int c) {
    int left_block = (l-1)/len, right_block = (r-1)/len;
    if (left_block == right_block) { // 同块暴力并重构
        for (int i=l; i<=r; ++i) a[i] += c;
        rebuild(left_block); // 将修改后的块重新排序
    } else {
        for (int i=l; i<= (left_block+1)*len; ++i) a[i] += c;
```

```
    rebuild(left_block);
    for (int i = left_block+1; i < right_block; ++i) lazy[i] += c;
    for (int i = right_block*len+1; i<=r; ++i) a[i] += c;
    rebuild(right_block);
}
}
```

(注：选手多次 WA 在于 `rebuild` 逻辑混乱或未正确处理懒标记与有序块的相对关系。)

f) 推荐同类题

- **P2801 教主的魔法**：与本题几乎相同，要求统计区间内大于等于某个值的个数，是非常好的强化练习。
- **P3870 [TJOI2009] 开关**：同属分块思想维护区间翻转与查询，难度略低，检验基础。

【报告结束】